Formal ILP Formulation for RunSoC 2.0

1 Problem Definition

This document formalizes the integer linear programming (ILP) model used for assigning and scheduling a set of tasks on a heterogeneous multi-core platform. The model minimizes a weighted objective consisting of the schedule makespan, memory-budget overflow penalties, and communication penalties between dependent tasks.

2 Sets

$$
\begin{aligned}
T &:= \text{set of tasks},\\
C &:= \text{set of cores},\\
K &:= \text{set of clusters},\\
D &\subseteq T \times T := \text{set of precedence dependencies},\\
E_i &\subseteq C := \text{set of eligible cores for task } i \in T,\\
C_k &:= \{c \in C \mid c \text{ belongs to cluster } k\}, \quad k \in K.
\end{aligned}
$$

A dependency $(i, j) \in D$ means that task i must finish before task j may start.

3 Parameters

For each task $i \in T$:

$$
\begin{aligned}
r_i &:= \text{minimum start time of task } i,\\
p_i &:= \text{base processing duration of task } i,\\
m_i &:= \text{memory demand of task } i.
\end{aligned}
$$

For each core $c \in C$:

$$
\begin{aligned}
\alpha_c &:= \text{WCET scaling factor of core } c,\\
B_c^{\text{core}} &:= \text{memory budget of core } c.
\end{aligned}
$$

For each cluster $k \in K$:

$$
B_k^{\text{cluster}} := \text{memory budget of cluster } k.
$$

Let $\kappa(c) \in K$ denote the cluster containing core c.
The processing time of task i on core c is

$$
p_{ic} := p_i \alpha_c. \tag{1}
$$

The model uses a sufficiently large constant M.

In the implementation, the maximal achievable horizon is used:

$$M = \max_{i \in T} r_i + \sum_{i \in T} \max_{c \in E_i} p_i \alpha_c + 1. \tag{2}$$

Memory-overflow penalty weights are denoted by

$$\begin{aligned} \lambda_{\text{core}} &:= \text{penalty weight for core-memory overflow,} \\ \lambda_{\text{cluster}} &:= \text{penalty weight for cluster-memory overflow.} \end{aligned}$$

Communication penalty weights are denoted by

$$\begin{aligned} w_{\text{intra}} &:= \text{penalty for communication on the same core,} \\ w_{\text{intercore}} &:= \text{penalty for communication between different cores in the same cluster,} \\ w_{\text{intercluster}} &:= \text{penalty for communication between cores in different clusters.} \end{aligned}$$

For an explicitly defined communication path (c, d), let

$$q_{cd}^{\text{explicit}} \tag{3}$$

be its specified penalty. The effective communication penalty between cores $c, d \in C$ is

$$q_{cd} = \begin{cases} q_{cd}^{\text{explicit}}, & \text{if an explicit communication path } (c,d) \text{ is defined,} \\ w_{\text{intra}}, & \text{if } c = d, \\ w_{\text{intercore}}, & \text{if } c \neq d \text{ and } \kappa(c) = \kappa(d), \\ w_{\text{intercluster}}, & \text{if } \kappa(c) \neq \kappa(d). \end{cases} \tag{4}$$

4 Decision Variables

$x_{ic} \in \{0,1\}$	$\forall i \in T,\ c \in C,$	equals 1 iff task i is assigned to core c,
$s_i \geq 0$	$\forall i \in T,$	start time of task i,
$f_i \geq 0$	$\forall i \in T,$	finish time of task i,
$C_{\max} \geq 0$		schedule makespan,
$o_c^{\text{core}} \geq 0$	$\forall c \in C,$	memory overflow on core c,
$o_k^{\text{cluster}} \geq 0$	$\forall k \in K,$	memory overflow on cluster k.

For each unordered pair of distinct tasks $\{i, j\}$ with $i < j$:

$y_{ij} \in \{0,1\}$ $\quad \forall i, j \in T,\ i < j,$ ordering variable used for non-overlap constraints.

For each dependency $(i, j) \in D$ and eligible core pair $c \in E_i$, $d \in E_j$:

$z_{ijcd} \in \{0,1\}$ $\quad \forall (i,j) \in D,\ c \in E_i,\ d \in E_j,$ equals 1 iff i is assigned to c and j is assigned to d.

5 Objective Function

The objective minimizes the makespan, weighted core-memory overflow, weighted cluster-memory overflow, and communication penalties along dependency edges:

$$\min \quad C_{\max} + \lambda_{\text{core}} \sum_{c \in C} o_c^{\text{core}} + \lambda_{\text{cluster}} \sum_{k \in K} o_k^{\text{cluster}} + \sum_{(i,j) \in D} \sum_{c \in E_i} \sum_{d \in E_j} q_{cd} z_{ijcd}. \tag{5}$$

6 Constraints

6.1 Assignment Constraints

Each task must be assigned exactly once to one of its eligible cores:

$$\sum_{c \in E_i} x_{ic} = 1 \qquad \forall i \in T. \tag{6}$$

Assignments to ineligible cores are forbidden:

$$x_{ic} = 0 \qquad \forall i \in T,\ \forall c \in C \setminus E_i. \tag{7}$$

6.2 Timing Constraints

Each task may not start before its minimum release time:

$$s_i \geq r_i \qquad \forall i \in T. \tag{8}$$

The finish time of each task is determined by its start time and the processing time induced by the selected core:

$$f_i = s_i + \sum_{c \in C} p_i \alpha_c x_{ic} \qquad \forall i \in T. \tag{9}$$

The makespan must be at least the finish time of every task:

$$C_{\max} \geq f_i \qquad \forall i \in T. \tag{10}$$

6.3 Precedence Constraints

For every dependency edge, the successor task may only start after the predecessor task has finished:

$$s_j \geq f_i \qquad \forall (i,j) \in D. \tag{11}$$

6.4 Memory-Budget Constraints

Core-level memory use may exceed the core memory budget only through the non-negative overflow variable:

$$\sum_{i \in T} m_i x_{ic} \leq B_c^{\text{core}} + o_c^{\text{core}} \qquad \forall c \in C. \tag{12}$$

Cluster-level memory use may exceed the cluster memory budget only through the non-negative overflow variable:

$$\sum_{i \in T} \sum_{c \in C_k} m_i x_{ic} \leq B_k^{\text{cluster}} + o_k^{\text{cluster}} \qquad \forall k \in K. \tag{13}$$

6.5 Non-Overlap Constraints on Cores

For every unordered pair of tasks $i, j \in T$ with $i < j$, and for every core $c \in C$, the model enforces that if both tasks are assigned to the same core, then one must finish before the other starts.

$$s_j \geq f_i - M\,(3 - y_{ij} - x_{ic} - x_{jc}) \qquad \forall i,j \in T,\ i < j,\ \forall c \in C. \tag{14}$$

$$s_i \geq f_j - M\,(2 + y_{ij} - x_{ic} - x_{jc}) \qquad \forall i,j \in T,\ i < j,\ \forall c \in C. \tag{15}$$

If $x_{ic} = x_{jc} = 1$, these constraints reduce to:

$$s_j \geq f_i - M(1 - y_{ij}),$$
$$s_i \geq f_j - M y_{ij}.$$

Thus, $y_{ij} = 1$ selects the ordering i before j, while $y_{ij} = 0$ selects the ordering j before i.

6.6 Communication-Penalty Linearization

For each dependency $(i, j) \in D$ and each eligible core pair $c \in E_i$, $d \in E_j$, the auxiliary variable z_{ijcd} represents the conjunction

$$z_{ijcd} = x_{ic} x_{jd}.$$

This product is linearized using:

$$z_{ijcd} \leq x_{ic} \qquad \forall (i,j) \in D,\ c \in E_i,\ d \in E_j, \tag{16}$$
$$z_{ijcd} \leq x_{jd} \qquad \forall (i,j) \in D,\ c \in E_i,\ d \in E_j, \tag{17}$$
$$z_{ijcd} \geq x_{ic} + x_{jd} - 1 \qquad \forall (i,j) \in D,\ c \in E_i,\ d \in E_j. \tag{18}$$

Since z_{ijcd} is binary, these constraints force $z_{ijcd} = 1$ exactly when both $x_{ic} = 1$ and $x_{jd} = 1$.

7 Variable Domains

$$\begin{aligned}
x_{ic} &\in \{0,1\} && \forall i \in T,\ c \in C,\\
y_{ij} &\in \{0,1\} && \forall i,j \in T,\ i < j,\\
z_{ijcd} &\in \{0,1\} && \forall (i,j) \in D,\ c \in E_i,\ d \in E_j,\\
s_i, f_i &\in [0, M] && \forall i \in T,\\
C_{\max} &\in [0, M], &&\\
o_c^{\text{core}} &\geq 0 && \forall c \in C,\\
o_k^{\text{cluster}} &\geq 0 && \forall k \in K.
\end{aligned}$$

8 Remarks

The current implementation models task eligibility through the input set E_i for each task. Hence, constraints related to task type, execution domain, or memory-node accessibility are assumed to be preprocessed into the eligible-core sets and are not explicitly represented as separate ILP constraints.

The communication objective is evaluated only over precedence dependencies. Therefore, a communication penalty is incurred for a task pair (i, j) only if $(i, j) \in D$.

Memory budgets are modeled as soft constraints. Violations are permitted through overflow variables, but are penalized in the objective function. Consequently, the model remains feasible even when strict core-level or cluster-level memory budgets cannot be satisfied.